

Satisfiability and SMT

Verifying cyberphysical systems

Sayan Mitra

mitras@illinois.edu

Some of the slides for this lecture are adapted from slides by Clark Barrett

Boolean *satisfiability* problem

Given a *well-formed formula* in propositional logic, determine whether there exists a satisfying solution

Example: $\alpha(x_1, x_2, \dots, x_n) \equiv (x_1 \wedge x_2 \vee x_3) \wedge (x_1 \wedge \neg x_3 \vee x_2)$

Set of variables: $X = \{x_1, x_2, \dots, x_n\}$,

Each variable is Boolean: $type(x_i) = \{0,1\}$

Formula α is *well-formed* if it uses propositional operators, and $\wedge$, or $\vee$, not $\neg$, iff $\leftrightarrow$ etc., properly

Recall, a valuation $\mathbf{x}$ of X maps each x_i to a value 0 or 1

A valuation $\mathbf{x}$ of X *satisfies* α if each each x_i in α replaced by the corresponding value in $\mathbf{x}$ evaluates to *true*. We write this as $\mathbf{x} \models \alpha$

Otherwise, we write $\mathbf{x} \not\models \alpha$

Example: with $\mathbf{x} \equiv \langle x_1 \mapsto 1, x_2 \mapsto 1, x_3 \mapsto 0 \rangle$; $\mathbf{x} \models \alpha$

Boolean *satisfiability* problem (SAT)

Given a well-formed formula in propositional logic, determine whether there exists a satisfying solution

Restatement: $\exists \mathbf{x} \in \text{val}(X): \mathbf{x} \models \alpha$?

If the answer is "No" then α is said to be *unsatisfiable*

Aside. If $\forall \mathbf{x} \in \text{val}(X): \mathbf{x} \models \alpha$ then α is said to be *valid* or *a tautology*

If α is valid then $\neg \alpha$ is unsatisfiable

α and α' are *tautologically equivalent* if they have the same truth tables

$$\forall \mathbf{x} \in \text{val}(X): \mathbf{x} \models \alpha \leftrightarrow \mathbf{x} \models \alpha'$$

What is a naïve method for solving SAT?

What is the complexity of this approach? How many evaluations of $\alpha(x_1, x_2, \dots, x_n)$?

SAT is NP-complete

SAT was the first problem shown to be NP-complete [Cook 71]

2-SAT can be solved in polynomial time (Exercise)

(Read definition of NP: Nondeterministic Polytime in Appendix C)

This has real implications

1. Essentially we don't know better than the naïve algorithm
2. A solver for SAT can be used to solve any other problem in the NP class with only polytime slowdown. i.e., makes a lot of sense to build SAT solvers
3. SAT/SMT solving is the cornerstone of *many* verification procedures

Stephen Cook, The complexity of theorem-proving procedures. In Proceedings of the third annual ACM symposium on theory of computing. STOC '71.

Online SAT solvers [edit]

- BoolSAT – Solves formulas in the DIMACS-CNF format or in a more
- Logictools – Provides different solvers in javascript for learning, co
- minisat-in-your-browser – Solves formulas in the DIMACS-CNF for
- SATRennesPA – Solves formulas written in a user-friendly way. R
- somerby.net/mack/logic – Solves formulas written in symbolic logic

Offline SAT solvers [edit]

- MiniSAT – DIMACS-CNF format and OPB format for it's companion
- Lingeling – won a gold medal in a 2011 SAT competition.
 - PicoSAT – an earlier solver from the Lingeling group.
- Sat4j – DIMACS-CNF format. Java source code available.
- Glucose – DIMACS-CNF format.
- RSat – won a gold medal in a 2007 SAT competition.
- UBCCSAT. Supports unweighted and weighted clauses, both in the
- CryptoMiniSat – won a gold medal in a 2011 SAT competition. C++ MiniSat 2.0 core, PrecoSat ver 236, and Glucose into one package, .
- Spear – Supports bit-vector arithmetic. Can use the DIMACS-CNF
 - HyperSAT – Written to experiment with B-cubing search space solver from the developers of Spear.
- BASolver
- ArgoSAT
- Fast SAT Solver – based on genetic algorithms.
- zChaff – not supported anymore.

The international SAT Competitions web page

Current Competition											
		SAT 2019 Race									
Organizers		Marlin Heule, Matti Järvinen, Martin Suda									
Past Competitions											
		SAT 2018 Competition									
Organizers		Marlin Heule, Matti Järvinen, Martin Suda									
		SAT 2017 Competition									
Organizers		Marlin Heule, Matti Järvinen, Tomáš Balyo									
Slides		Slides used at SAT 2017									
Proceedings		Descriptions of the solvers and benchmarks									
Benchmarks		Available here									
Solvers		Available here									
		Gold		Silver		Bronze		Gold		Silver	
SAT+UNSAT		CaDiCal, Agile, CaDiCal, NoProof		Glu_VC		Glucose 4.1		Maple LCM Dist, Maple LCM, Maple RB LCM, Maple RB LCM		Main Track	
		Parallel Track									
SAT+UNSAT		Synq24, Synq48		Plingling		Fairness MapleCOMSPS		COMINSATPS Pular		No-Limit Track	
		MapleCOMSPS LRB VSIDS 2, MapleCOMSPS LRB VSIDS 3									
SAT 2016 Competition											
Organizers		Marlin Heule, Matti Järvinen, Tomáš Balyo									
Proceedings		Descriptions of the solvers and benchmarks									
Benchmarks		Available here									
Solvers		Available here									
		Gold		Silver		Bronze		Gold		Silver	
				Agile Track						Main Track	

Details

We will assume α to be in *conjunctive normal form (CNF)*

literals: variable or its negation, e.g., x_3 , $\neg x_3$

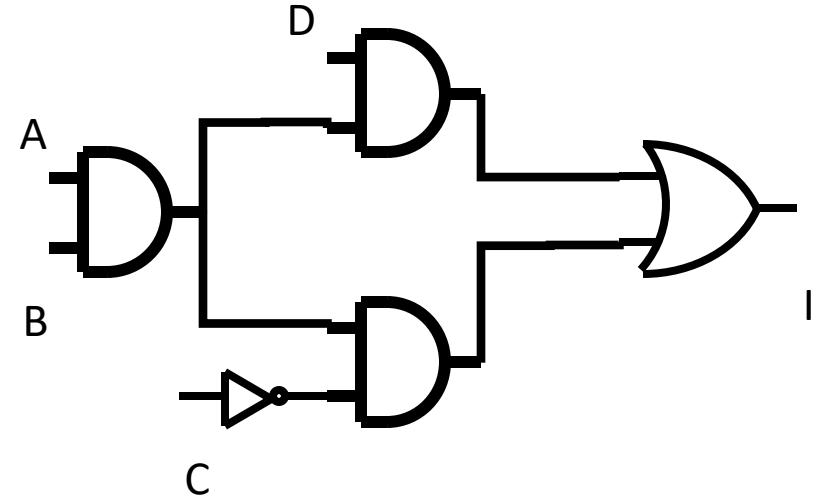
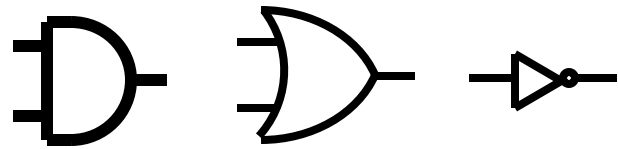
clause: disjunction (or) of literals, e.g., $(x_1 \vee x_2 \vee \neg x_3)$

CNF formula: conjunction (and) of clauses,

e.g., $(x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_2 \vee x_1)$

A variable may appear *positively* or *negatively* in a clause

Logic and circuits



$$I \equiv (D \wedge (A \wedge B)) \vee (\neg C \wedge (A \wedge B))$$

Repeated subexpression is inefficient

Solution: rename $(A \wedge B) \leftrightarrow E$

$$I' \equiv (D \wedge E) \vee (\neg C \wedge E) \wedge ((A \wedge B) \leftrightarrow E)$$

I and I' are **not** *tautologically equivalent*

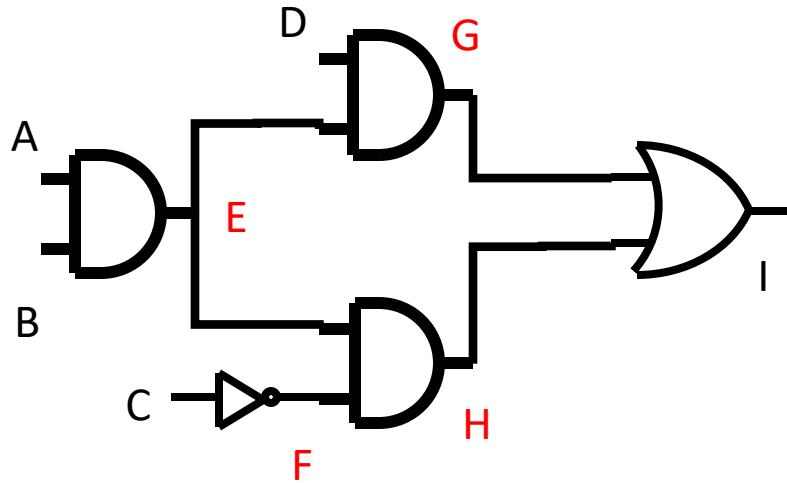
$C = 0, A = B = 1, E = 0$ satisfies I

But they are *equisatisfiable*, i.e., I is satisfiable iff I' is also satisfiable

Converting to CNF

- View the formula as a graph
- Give new names (variables) to non-leafs
- Relate the inputs and the outputs of the nonleafs and add this as a new clause
- Take conjunction of all of this

Converting to CNF



- $F \leftrightarrow \neg C$
 - $F \rightarrow \neg C \wedge \neg C \rightarrow F$
 - $(\neg F \vee C) \wedge (\neg C \vee F)$
- $(A \wedge B) \leftrightarrow E$
 - $((A \wedge B) \rightarrow E) \wedge (E \rightarrow (A \wedge B))$
 - $(\neg(A \wedge B) \vee E) \wedge (\neg E \vee (A \wedge B))$
 - $(\neg A \vee \neg B \vee E) \wedge (\neg E \vee A) \wedge (\neg E \vee B)$
- $(G \vee H) \leftrightarrow I$
 - $((G \vee H) \rightarrow I) \wedge (I \rightarrow (G \vee H))$
 - $(\neg G \wedge \neg H \vee I) \wedge (\neg I \vee G \vee H)$
 - $(\neg G \vee I) \wedge (\neg H \vee I) \wedge (\neg I \vee G \vee H)$
- $(D \wedge E) \leftrightarrow G$
 - $(\neg D \vee \neg E \vee G) \wedge (\neg G \vee D) \wedge (\neg G \vee E)$
- $(F \wedge E) \leftrightarrow H$
 - $(\neg F \vee \neg E \vee H) \wedge (\neg H \vee F) \wedge (\neg H \vee E)$

Standard representations of CNF

- $(\neg A \vee \neg B \vee E) \wedge (\neg E \vee A) \wedge (\neg E \vee B)$
- $(A' + B' + E)(E' + A)(E' + B)$
- $(-1 \ -2 \ 5)(-5 \ 1)(-5 \ 2)$ DIMACS
- SMTLib

Davis Putnam Logemann Loveland Algorithm (DPLL) 1962

Transform the given formula α by applying a sequence of satisfiability preserving rules

If final result has an empty clause then *unsatisfiable*

if final result has no clauses then the formula is *satisfiable*

Davis Putnam Algorithm (DP) 1960

Rule 1. **Unit propagation**

Rule 2. **Pure literal**

Rule 3. **Resolution**

DP 1960

Rule 1. **Unit propagation**

A clause has a single literal

$$\alpha \equiv \cdots \wedge \cdots \wedge p \wedge \cdots \wedge \cdots$$

What choice do we really have?

$$\alpha \equiv \cdots \wedge (x_1 \vee \neg p \vee x_2) \wedge p \wedge \cdots \wedge (\neg x_3 \vee \neg p \vee x_1) \dots$$

DP 1960

Rule 1. **Unit propagation**

A clause has a single literal

$$\alpha \equiv \cdots \wedge \cdots \wedge p \wedge \cdots \wedge \cdots$$

What choice do we really have?

$$\alpha' \equiv \cdots \wedge (x_1 \vee x_2) \wedge \cdots \wedge (\neg x_3 \vee x_1) \dots$$

α and α' are equisatisfiable

Davis Putnam Logemann Loveland Algorithm (DPLL) 1962

Rule 1. **Unit propagation**

Rule 2. **Pure literal**

A literal appears only positively (or negatively) in α

$$\alpha \equiv \cdots \wedge (x_1 \vee \neg p \vee x_2) \wedge (x_4 \vee \neg p) \wedge \cdots \wedge (\neg x_3 \vee \neg p \vee x_1) \dots$$

p does not appear anywhere

Makes sense to set $p = 0$ and remove all occurrences of $\neg p$

Davis Putnam Logemann Loveland Algorithm (DPLL) 1962

Rule 1. **Unit propagation**

Rule 2. **Pure literal**

A literal appears only positively (or negatively) in α

$$\alpha \equiv \cdots \wedge (x_1 \vee \neg p \vee x_2) \wedge (x_4 \vee \neg p) \wedge \cdots \wedge (\neg x_3 \vee x_1) \dots$$

p does not appear anywhere

Makes sense to set $p = 0$ and remove all clauses in which $\neg p$ occurs

α and α' are equisatisfiable

$$\alpha' \equiv \cdots \wedge \cdots \wedge \cdots \wedge (\neg x_3 \vee x_1) \dots [p = 0]$$

Davis Putnam Algorithm (DP) 1960

Rule 1. **Unit propagation**

Rule 2. **Pure literal**

Rule 3. **Resolution**

Choose a literal p that appears with both polarity in α . Suppose $(\ell_1 \vee \ell_2 \vee \dots \vee p)$ be a clause in which p appears positively, and $(k_1 \vee k_2 \vee \dots \vee \neg p)$ be a clause in which p appears negatively

Then the resolved clause is $(\ell_1 \vee \ell_2 \vee \dots \vee k_1 \vee k_2 \vee \dots \vee k_m)$

Pairwise, resolve each clause in which p appears positively with a clause in which p appears negatively, and take the conjunction of all the results

Why is the result equisatisfiable?

What is the size of the resulting formula?

DPLL modifies resolution in DP with recursive DFS rule

Rule 1. **Unit propagation**

Rule 2. **Pure literal**

Rule 3'. Let Δ be the current set of clauses. Choose a literal p in Δ .

Check satisfiability of $\Delta \cup \{ p \}$ (guessing $p = 1$)

If satisfiable then return True else

return result of checking satisfiability of $\Delta \cup \{ \neg p \}$

This is essentially a depth first search

A simple greedy algorithm for SAT (GSAT)

Input: Set of clauses C over X , parameters *max-flips*, *max-tries*

Output: A satisfying assignment for C , or $\emptyset$ if none found

for $i = 1$ to *max-tries*

$v :=$ random truth assignment in $val(X)$

 for $j = 1$ to *max-flips*

 if $v \models C$ then return v

$p :=$ variable in C such that flipping its value gives the largest increase in the number of clauses of C that are satisfied by v

$v := v$ with the assignment to p flipped

return $\emptyset$

From Slides of Clark Barrett's
lecture.
Summer School on Verification
Technology, Systems & Applications,
September 17, 2008 – p. 42/98

Problem	tautology	dptaut	dplltaut
prime 3	0.00	0.00	0.00
prime 4	0.02	0.06	0.04
prime 9	18.94	2.98	0.51
prime 10	11.40	3.03	0.96
prime 11	28.11	2.98	0.51
prime 16	>1 hour	out of memory	9.15
prime 17	>1 hour	out of memory	3.87
ramsey 3 3 5	0.03	0.06	0.02
ramsey 3 3 6	5.13	8.28	0.31
mk_adder_test 3 2	>>1 hour	6.50	7.34
mk_adder_test 4 2	>>1 hour	22.95	46.86
mk_adder_test 5 2	>>1 hour	44.83	170.98
mk_adder_test 5 3	>>1 hour	38.27	250.16
mk_adder_test 6 3	>>1 hour	out of memory	1186.4
mk_adder_test 7 3	>>1 hour	out of memory	3759.9

Stålmarck's algorithm

Breadth first search instead of depth-first

Given a set of clauses Δ and any basic deduction algorithm R , the **dilemma rule** performs a case split on some literal p by considering the new sets of clauses $\Delta \cup \{(\neg p)\}$ and $\Delta \cup \{(p)\}$.

R is applied to each of these to get Δ_0 and Δ_1 respectively

The original Δ is augmented with $\Delta_0 \cap \Delta_1$

Abstract DPLL

- Abstract DPLL uses **states** and **transitions** to model the progress of the algorithm
- Most states are of the form $M||F$ where
 - M is a **sequence** of annotated **literals** denoting partial truth assignment
 - F is the CNF formula being checked, represented as a set of **clauses**
- **Initial state:** $\emptyset||F$, where F is to be checked for satisfiability
- Transitions between states are defined by a set of **conditional transition rules**
- **Final state**
 - *Fail* special state, if F is unsatisfiable, or
 - $M||G$, where G is CNF formula equisatisfiable with original F and $M \models G$
- We will write $M \models C$ to mean that every truth assignment v , $v(M) = \text{True}$ implies $v(C) = \text{True}$

Abstract DPLL

$\times$

$\times$

UnitProp: $M F, C \vee \ell$	$\rightarrow M \ell F, C \vee \ell$	If $\begin{cases} M \models \neg C \\ \ell \text{ is undefined in } M \end{cases}$
PureLiteral: $M F$	$\rightarrow M \ell F$	If $\begin{cases} \ell \text{ occurs in some clause of } F \\ \neg \ell \text{ occurs in no clause of } F \\ \ell \text{ is undefined in } M \end{cases}$
Decide: $M F$	$\rightarrow M \ell^d F$	If $\begin{cases} \ell \text{ or } \neg \ell \text{ occurs in a clause of } F \\ \ell \text{ is undefined in } M \end{cases}$
Backtrack: $M \ell^d N F, C$	$\rightarrow M \neg \ell F, C$	If $\begin{cases} M \ell^d N \models \neg C \\ N \text{ contains no decision literals} \end{cases}$
Fail: $M F, C$	$\rightarrow \text{fail}$	If $\begin{cases} M \models \neg C \\ M \text{ contains no decision literals} \end{cases}$

An Example: Abstract DPLL

$\phi || 1 \vee \bar{2} \quad \bar{1} \vee \bar{2} \quad 2 \vee 3 \quad \bar{3} \vee 2 \quad 1 \vee 4$

$\Rightarrow$ (PureLiteral)

$4 || 1 \vee \bar{2} \quad \bar{1} \vee \bar{2} \quad 2 \vee 3 \quad \bar{3} \vee 2 \quad \underline{1 \vee 4}$

$\Rightarrow$ (Decide)

$4 \ 1^d || \underline{1 \vee \bar{2}} \quad \bar{1} \vee \bar{2} \quad 2 \vee 3 \quad \bar{3} \vee 2 \quad 1 \vee 4$

$\Rightarrow$ (UnitProp)

$4 \ 1^d \ \bar{2} || 1 \vee \bar{2}. \quad \bar{1} \vee \bar{2} \quad 2 \vee 3 \quad \bar{3} \vee 2 \quad 1 \vee 4$

$\Rightarrow$ (UnitProp)

$4 \ 1^d \ \bar{2} \ 3 || 1 \vee \bar{2}. \quad \bar{1} \vee \bar{2} \quad 2 \vee 3 \quad \bar{3} \vee 2 \quad 1 \vee 4$

$4 \ 1^d \ \bar{2} \ 3 || 1 \vee \bar{2}. \quad \bar{1} \vee \bar{2} \quad 2 \vee 3 \quad \bar{3} \vee 2 \quad 1 \vee 4$

$\Rightarrow$ (Backtrack)

$4 \ \bar{1} || 1 \vee \bar{2} \quad \bar{1} \vee \bar{2} \quad 2 \vee 3 \quad \bar{3} \vee 2 \quad 1 \vee 4$

$\Rightarrow$ (UnitProp)

$4 \ \bar{1} \ \bar{2} || 1 \vee \bar{2} \quad \bar{1} \vee \bar{2} \quad 2 \vee 3 \quad \bar{3} \vee 2 \quad 1 \vee 4$

$\Rightarrow$ (UnitProp)

$4 \ \bar{1} \ \bar{2} \ 3 || 1 \vee \bar{2} \quad \bar{1} \vee \bar{2} \quad 2 \vee 3 \quad \bar{3} \vee 2 \quad 1 \vee 4 \Rightarrow \text{Fail}$

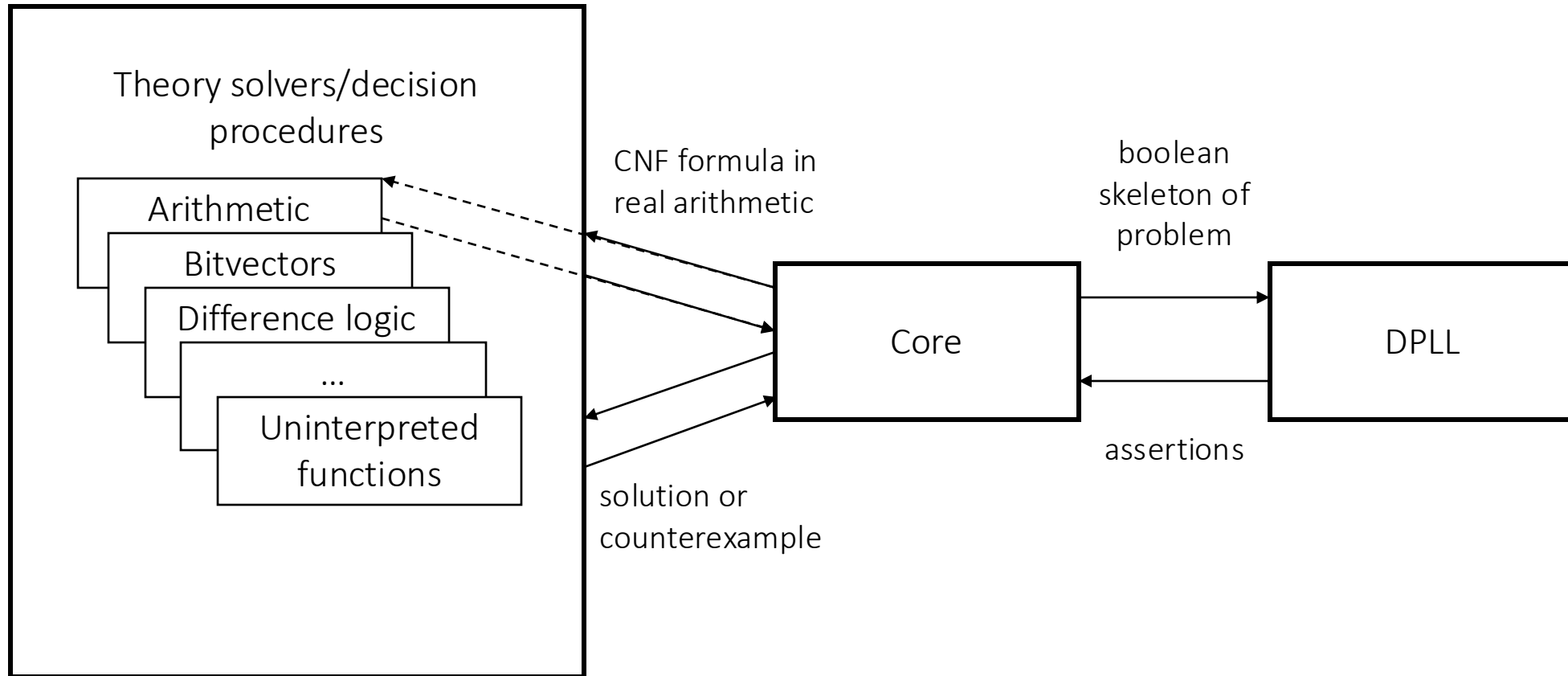
Satisfiability modulo theories

- SAT: Given a *well-formed formula* in propositional logic, determine whether there exists a satisfying solution
- A *satisfiability modulo theory (SMT)* problem is a generalization of SAT in which some of the binary variables are replaced by predicates over a suitable set of non-binary variables
- $\phi_1(w, x, y, z) := (x - y = 5) \wedge (z - y \geq 2) \wedge (z - x > 2) \wedge (w - x = 2)$
- $\phi_2(x, y, z) := (3x^2 - 4y + 5z \leq 5) \wedge (-2x + 5z^3 \leq 7)$
- ϕ_1 is a predicate in *difference logic* in which the variables are real-valued, and the clauses are constructed with standard comparison operations $>$, $\geq$, $=$ and $-$ (minus)
- ϕ_2 is a predicate in real arithmetic

SMT

A short overview of theories, models, decision procedures

Architecture of an SMT solver



What is a theory in mathematical logic?

- When we talk about **well-formed formulas with non-binary variables**, we have to say exactly what type of formulas are allowed
- and, what it means for assignments to *satisfy such formulas*
- This brings us to the notions **theory** and **models** in mathematical logic

Building up a theory

- First we define the syntax for writing formulas
 - A *signature* $\Sigma = (\Sigma_F, \Sigma_P, V)$
 - set of *function symbols* Σ_F , e. g., $\{+, -, f, g, \sin, \dots\}$
 - set of *predicate symbols* Σ_P
 - *arity* of each function: *arity*: $\Sigma_F \rightarrow \mathbb{N}$
 - 0 *arity* functions are constants
 - V : set of *variables*
 - *Terms*(Σ, V)
 - Elements of V are terms
 - If $t_1, \dots, t_k \in \text{Terms}(\Sigma, V)$ and $f \in \Sigma_F$ with arity k , then $f(t_1, \dots, t_k) \in \text{Terms}(\Sigma, V)$
 - *Ground terms* are terms without variables
- $\Sigma_F = \{0, +\}, \Sigma_P = \{<\}$
 - $\text{arity}(0) = 0$
 - $\text{arity}(+) = 2$
 - $\text{arity}(<) = 2$
 - $V = \{x, y, z\}$
 - Terms defined by this signature are $x, y, z, +(x, y), ++(x, y), 0, \dots$

Terms to Formulas

- **Atomic formulas** AF
 - True, False
 - If $t_1, \dots, t_k \in Terms(\Sigma, V)$ and $p \in \Sigma_P$ with arity k , then $p(t_1, \dots, t_k) \in AF(\Sigma, V)$
 - A *literal* is an AF or its negation
 - Set of all atomic formulas $AF(\Sigma, V)$
 - **Quantifier free formulas** $QFF(\Sigma, V)$
 - AF
 - if $\phi_1, \phi_2 \in QFF$ then
 - $\neg\phi_1 \in QFF, \phi_1 \wedge \phi_2 \in QFF, \phi_1 \vee \phi_2 \in QFF, \phi_1 \rightarrow \phi_2 \in QFF$
 - Set of all quantifier free formulas $QFF(\Sigma, V)$
 - **First order formulas** is the set of quantifier free formulas under universal and existential quantifiers
 - **Bound variables** are those that are attached to quantifiers
 - **Free variables**: variables not bound
 - **Sentence**: First order formula with no free variables
 - **Theory** (Σ, V) set of all sentences over (Σ, V)
- $x < y$
 - $+(x, y) = +(y, x)$
 - $+(x, y) = 0 \wedge x > y$
 - $\forall x, \exists y: +(x, y) = 0$
 - $\forall x, \exists y: x < y$
 - $\forall x, \exists y: +(x, y) = x$
 - $\exists x: +(x, c) = x$

Models for theories

This notion of model from mathematical logic is not to be confused with the notion of a model for a computational or physical process

- A *model* gives meanings or *interpretations* to formulas in theory T
- A model M for $T = \text{Theory}(\Sigma, V)$ has to define
 - A *domain* $|M|$
 - interpretations of all functions and predicate symbols
 - $M(f): |M|^n \rightarrow |M|$ if $\text{arity}(f) = n$
 - $M(p) \subseteq |M|^n$ if $\text{arity}(p) = n$
 - Assignment $M(x) \in |M|$ for every variable $x \in V$
- A *formula* ϕ is *true in* M if it evaluates to true under the given interpretations over domain M

Example model for $\Sigma = \{0, +, <\}$

$$|M| = \{a, b, c\}$$

$$M(0) = a$$

$$M(<) = \{\langle a, b \rangle, \langle a, c \rangle, \langle b, c \rangle\}$$

$M(+)$	a	b	b
a	a	b	c
b	b	c	a
c	c	a	b

□

$$\text{if } M(x) = a, M(y) = b$$

$$\text{then } M(+ (x, y)) \text{ is } M(+)(M(x), M(y)) = M(+)(a, b) = b$$

$$M(+ (+ (x, y), y)) = c$$

$$M \models \forall x \exists y + (x, y) = x$$

We say that the model M *T-satisfies* the formula ϕ

Decision procedures

Given a theory T a **theory solver or a decision procedure** for T takes as input a set of **literals** ϕ (atomic propositions) and determines whether ϕ is **T -satisfiable**, that is,

$\exists$ a model M such that $M \models \phi$?

$\langle \backslash \textit{logic} \rangle$

A short overview of theories and models in mathematical logic

Example theories

- Uninterpreted functions (UF) $\Sigma_F := \{f, g, \dots\}$, $\Sigma_P := \{=\}$, $V := \{x_i\}$
 - $x_1 = x_2 \wedge x_3 \neq x_2 \wedge f(x_3) \neq f(x_2)$
- Difference logic $\Sigma_F := \{1, 2, \dots, -\}$, $\Sigma_P := \{<, \leq, =, >, \geq\}$
 - $x_1 - x_2 \geq k$, where $\geq \in \{<, \leq, =, >, \geq\}$
- Linear arithmetic
 - $4x - 3y + 6z \leq 10$
- Real arithmetic (nonlinear)
 - $4x^2 + 6y - 9z^3 \leq 5$
- Bit vectors
- Arrays
 - $x'[i] = x[i] + 1$

Example decision procedure 1: Difference logic

$$\phi = (x - y = 5) \wedge (z - y \geq 2) \wedge (z - x > 2) \wedge (w - x = 2) \wedge (z - w < 0)$$

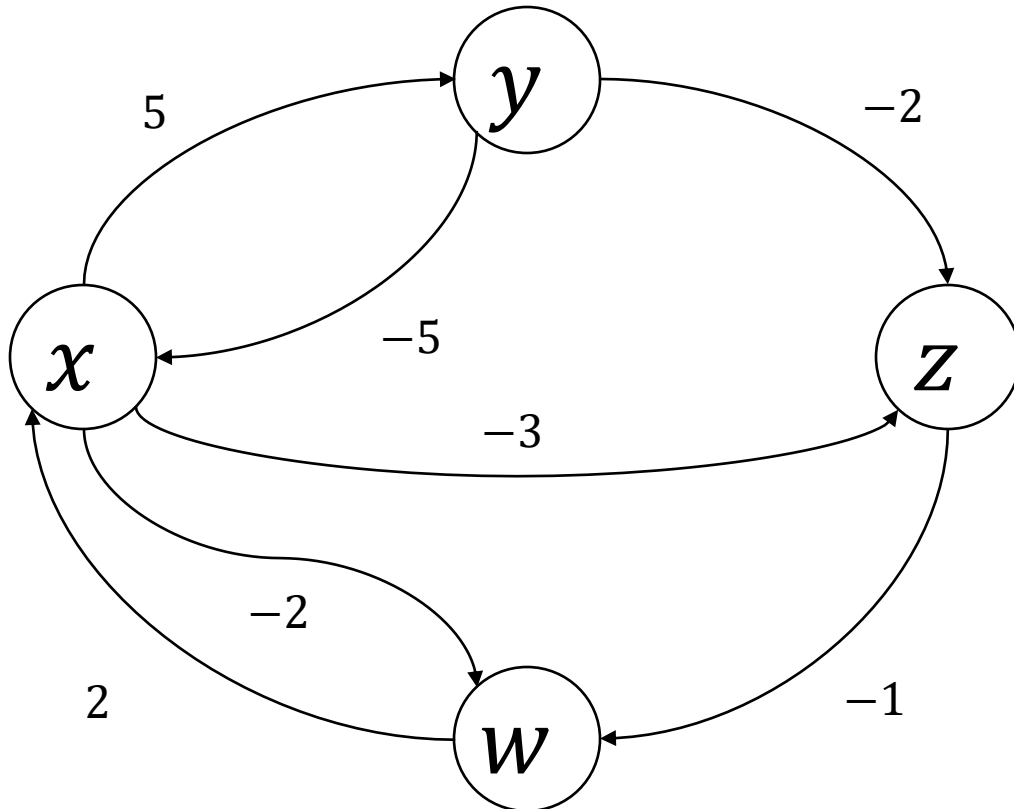
Decision procedure:

Convert each literal (AF) to $x_1 - x_2 \leq c$ form:

$$\begin{aligned}\phi' = & (x - y \leq 5) \wedge (y - x \leq -5) \\ & \wedge (y - z \leq -2) \wedge \\ & (x - z \leq -3) \wedge \\ & (w - x \leq 2) \wedge (x - w \leq -2) \\ & (z - w \leq -1)\end{aligned}$$

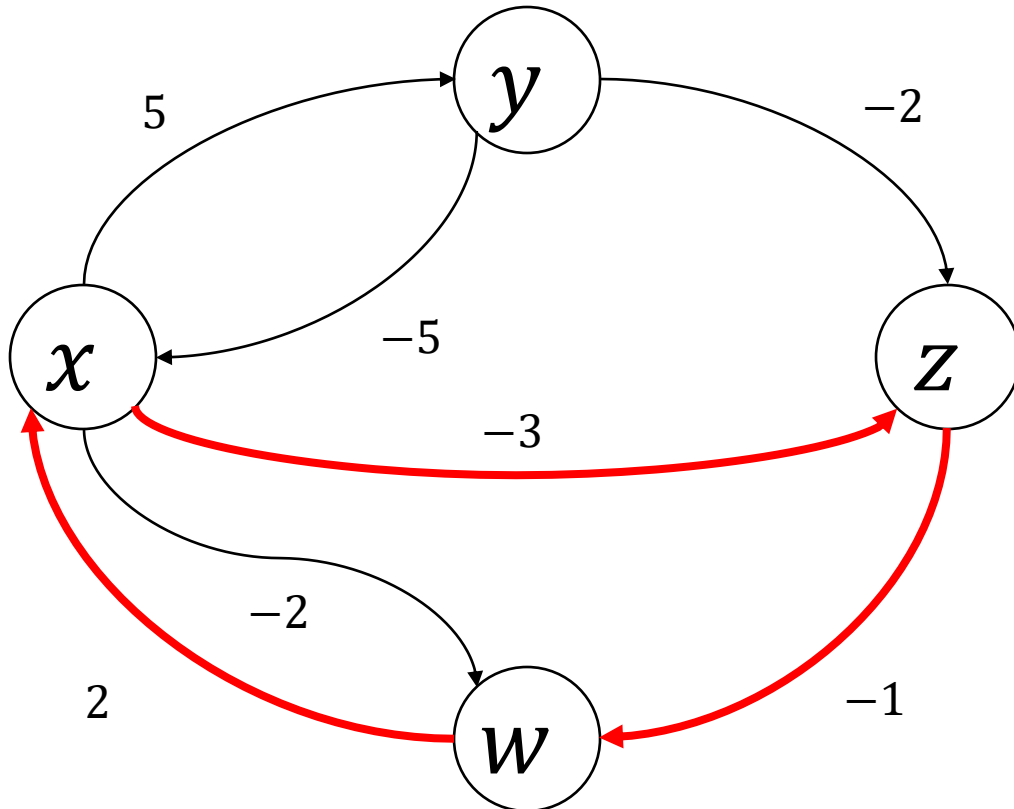
$$\begin{aligned}\phi' = & (x - y \leq 5) \wedge (y - x \leq -5) \\ & \wedge (y - z \leq -2) \wedge \\ & (x - z \leq -3) \wedge \\ & (w - x \leq 2) \wedge (x - w \leq -2) \\ & (z - w \leq -1)\end{aligned}$$

Construct a graph with edge from $x \rightarrow^c y$ for each literal ϕ'



$$\begin{aligned}\phi' = & (x - y \leq 5) \wedge (y - x \leq -5) \\ & \wedge (y - z \leq -2) \wedge \\ & (x - z \leq -3) \wedge \\ & (w - x \leq 2) \wedge (x - w \leq -2) \\ & (z - w \leq -1)\end{aligned}$$

Construct a graph $G_{\phi'}$, with edge from $x \rightarrow^c y$ for each literal ϕ'



Proposition. ϕ is satisfiable iff $G_{\phi'}$ is negative cycle free.

Exercise.

Example decision procedure 2: Uninterpreted functions (UF)

$$\phi = x_1 = x_2 \wedge (x_2 = x_3) \wedge (x_4 = x_5) \wedge (x_5 \neq x_1) \wedge (F(x_1) \neq F(x_3))$$

Decision procedure

1. Put all variables and function instances in their own classes
2. If $t_1 = t_2$ is a literal then merge the classes containing them; do this repeatedly
3. If t_1 and t_2 are terms in the same class then merge classes containing $F(t_1)$ and $F(t_2)$; repeat
4. If $t_1 \neq t_2$ is a literal in ϕ and they belong to the same class then return unsat else return sat t_1 and t_2

Example decision procedure 2: Uninterpreted functions (UF)

Initial classes $\phi = x_1 = x_2 \wedge (x_2 = x_3) \wedge (x_4 = x_5) \wedge (x_5 \neq x_1) \wedge (F(x_1) \neq F(x_3))$

Classes $\{x_1\} \{x_2\} \{x_3\} \{x_4\} \{x_5\} \{F(x_1)\} \{F(x_3)\}$

$\{x_1, x_2, x_3\} \{x_4, x_5\} \{F(x_1)\} \{F(x_3)\}$

$\{x_1, x_2, x_3\} \{x_4, x_5\} \{F(x_1), F(x_3)\}$

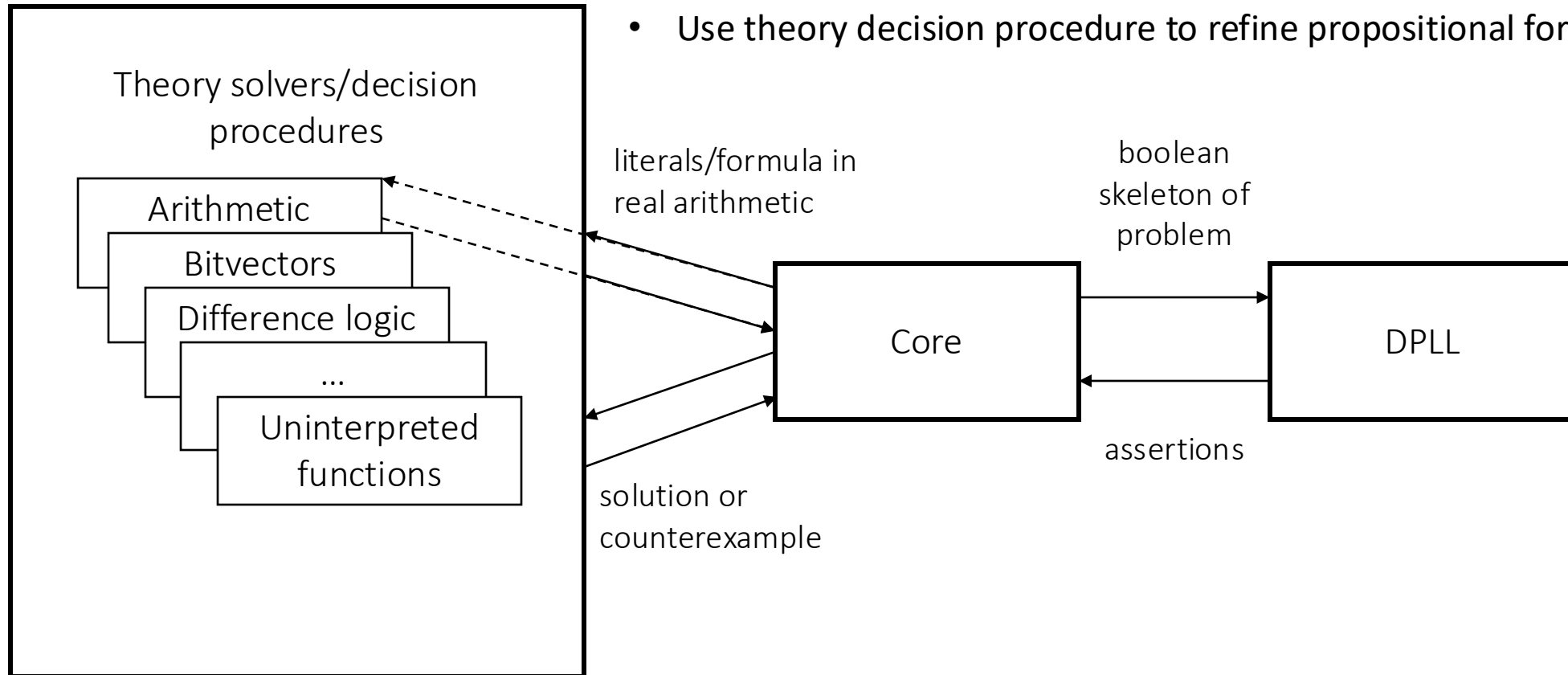
Unsat

Return to SMT

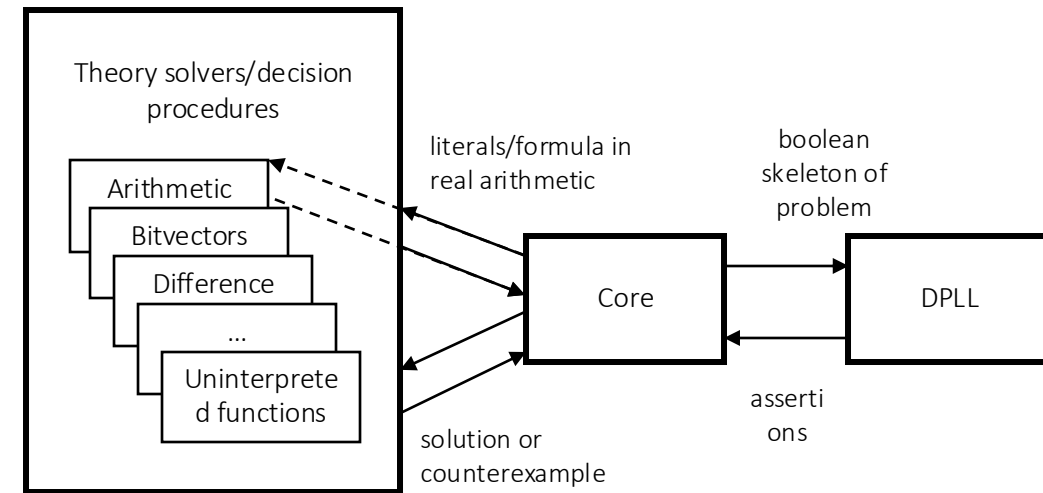
$$\phi \equiv g(a) = c \wedge f(g(a)) \neq f(c) \vee g(a) = d \wedge c \neq d$$

Several approaches, lazy approach:

- Abstract ϕ to propositional form
- Feed to DPLL
- Use theory decision procedure to refine propositional formula a guide SAT



$$\phi \equiv \underbrace{g(a) = c}_1 \wedge \underbrace{f(g(a)) \neq f(c)}_{\bar{2}} \vee \underbrace{g(a) = d}_3 \wedge \underbrace{c \neq d}_{\bar{4}}$$



- send $\{1, \bar{2} \vee 3, \bar{4}\}$ to DPLL
- returns model $\{1, \bar{2}, \bar{4}\}$
- UF solver concretizes to $g(a) = c, f(g(a)) \neq f(c), c \neq d$
- checks this as UNSAT
- send $\{1, \bar{2} \vee 3, \bar{4}, \bar{1} \vee 2 \vee 4\}$ to DPLL
- returns model $\{1, 2, 3, \bar{4}\}$
- UF solver concretizes and finds this to be UNSAT
- send $\{\bar{1}, \bar{2} \vee 3, \bar{4}, \bar{1} \vee 2 \vee 4, \bar{1} \vee \bar{2} \vee \bar{3} \vee 4\}$ to DPLL
- returns UNSAT

Assignments

- HW1
- Learn z3
 - <https://ericpony.github.io/z3py-tutorial/guide-examples.htm>

Readings

- Chapter 7.5.3 of CPSBook on using SAT/SMT for verification
- Read chapter 4 for next week
- Reading more about decision procedures

